Proyecto U1

Asignatura: Desarrollo Móvil Integral
Docente: Ing. Froylán Alanís
Integrantes: Alexis Jiménez Salas, Melani Judith Zermeño Yáñez
Grupo y Grado : 10 D
Fecha: 5 de octubre de 2026

Indice

ETAPA 1 — Análisis, metodología y arquitectura 3
Problemática 3
Justificación 3
1.3 Objetivo general 4
Objetivos específicos 4
2. Usuarios 4
Administrador 4
Empleado 5
3. Alcance 5
Incluye 5
No incluye inicialmente 6
4. Requerimientos 6
4.1 Requerimientos funcionales 6
4.2 Requerimientos no funcionales 7
5. Historias de usuario 8
HU01 — Registro 8
HU02 — Inicio de sesión 8
HU03 — Gestión de productos 8
HU04 — Consulta de inventario 8
HU05 — Entrada de productos 8
HU06 — Salida de productos 8
HU07 — Historial 8
HU08 — Administración de usuarios 8
HU09 — Perfil 9
HU10 — API externa 9
6. Criterios de aceptación 9
7. Product Backlog 10
8. Metodología 11
Scrum 11
Roles 11
Sprints propuestos 11
9. Arquitectura 12
Arquitectura general 12
Componentes 13
Comunicación 14
10. Responsabilidades de cada capa 15
Capa de presentación 15
Capa de lógica de negocio 15
Capa de datos 15
ETAPA 2 — Patrones, framework y versionamiento 15
Patrones de diseño 16
Repository Pattern 16
Singleton 17
Framework de desarrollo móvil 17
Framework seleccionado 17
Lenguaje 17
Plataforma 17
Lenguaje 17
Capacidades 17
Integración 18
Comunidad y documentación 18
Adecuación al proyecto 19
Versionamiento 19
Repositorio 19
Estrategia de ramas 20

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE

main 20
develop 20
Ramas de funcionalidad 20
Convención de commits 21
Estrategia de integración 23
Relación con Scrum 23
Resumen de decisiones técnicas 24
ETAPA 3 — Seguridad, APIs y nube 24
Seguridad 24
Información sensible 24
Amenazas identificadas 25
Vulnerabilidades 25
Mecanismos de protección 26
Autenticación 26
Autorización por roles 26
HTTPS 27
Protección de contraseñas 27
Validación de datos 27
Protección contra SQL Injection 27
Variables de entorno 27
Servicios Cloud 27
Datos almacenados en la nube 28
API de terceros 28
Función dentro de StockGo 29
Justificación de la API externa 29
Comunicación entre los componentes 30
Flujo de autenticación 30
Flujo de registro de producto mediante API 31
Relación con la arquitectura del proyecto 32

ETAPA 1 — Análisis, metodología y arquitectura

Problemática

En pequeños negocios minoristas, tales como la abarrotera local "Abarrotes San José", el control de entradas, salidas y existencias de mercancía se realiza actualmente de forma manual mediante notas en libreta y hojas de cálculo desactualizadas. Esta práctica genera discrepancias constantes entre el inventario físico y el registrado, pérdida de tiempo en el rastreo de productos y la imposibilidad de auditoría al no contar con un registro exacto de qué empleado realizó cada movimiento. La falta de un sistema centralizado e interactivo dificulta la toma de decisiones oportunas para el reabastecimiento de mercancía.

Justificación

Se propone el desarrollo de StockGo, una aplicación móvil orientada a optimizar la gestión operativa de pequeños negocios. StockGo automatizará el registro de entradas y salidas de inventario, garantizará la trazabilidad de cada operación asociándola al usuario responsable y proporcionará visibilidad en tiempo real del stock disponible. Esto reducirá mermas, minimizará los errores humanos y optimizará el tiempo de atención al cliente,

permitiendo a la administración tomar decisiones de compra basadas en información precisa.

1.3 Objetivo general

Desarrollar una aplicación móvil cliente-servidor que automatice y optimice la gestión de inventario de un pequeño negocio en tiempo real, garantizando la trazabilidad de las operaciones y el control de accesos basado en roles.

Objetivos específicos

1. Implementar un módulo de autenticación y autorización seguro para diferenciar el acceso y permisos entre el rol Administrador y el rol Empleado.
2. Desarrollar la gestión integral (CRUD) de productos y la actualización automática del stock en respuesta a registros de entradas y salidas. }
3. Registrar la trazabilidad completa de cada movimiento de inventario, vinculando fecha, hora, cantidad y el ID del usuario responsable.
4. Integrar un servicio web (API REST) en la nube y una API de terceros (por ejemplo, lector de código de barras / Open Food Facts) para agilizar la captura de productos.
5. Validar el correcto funcionamiento mediante pruebas unitarias y de integración, generando finalmente el ejecutable compilado en versión release.

2. Usuarios

La aplicación contará con **dos tipos de usuario**, cumpliendo uno de los requisitos mínimos del proyecto. PROYECTO INTEGRADOR DMI (1)

Administrador

Perfil: Dueño o gerente del establecimiento. Posee conocimientos básicos de navegación en smartphones.

Responsabilidades: Gestión total del catálogo de productos, auditoría del historial de movimientos, administración del personal (creación, edición y revocación de cuentas de empleados) y configuración del sistema.

Flujo de Registro/Acceso: El primer usuario Administrador se registra directamente desde la aplicación. Los administradores posteriores o cuentas de empleados son creadas y autorizadas exclusivamente por un Administrador previamente autenticado.

Puede:

- Registrarse e iniciar sesión.
- Consultar el inventario.
- Registrar productos.
- Modificar productos.
- Eliminar productos.
- Registrar entradas.
- Registrar salidas.
- Consultar el historial de movimientos.

- Administrar usuarios.
- Modificar su perfil.

Empleado

Será el usuario encargado de realizar operaciones relacionadas con el inventario.

Perfil: Personal operativo de caja o almacén. Requiere una interfaz intuitiva con acciones rápidas para evitar demoras operativas.

Responsabilidades: Consultar la disponibilidad de productos en tiempo real y registrar movimientos de entrada (recepción de proveedor) y salida (venta o merma).

Flujo de Registro/Acceso: No cuenta con registro público; su cuenta debe ser provista por el Administrador.

Puede:

- Iniciar sesión.
- Consultar productos.
- Consultar existencias.
- Registrar entradas.
- Registrar salidas.
- Consultar movimientos.
- Modificar su perfil.

No puede:

- Eliminar productos.
- Administrar usuarios.

3. Alcance

Soporte Uninegocio (Single-tenant): En esta primera versión, la arquitectura de StockGo está diseñada explícitamente para gestionar el inventario de un solo negocio o sucursal.

Compilado Release: Incluye la configuración, firma digital y generación del paquete ejecutable (release APK/AAB) para pruebas de despliegue.

Incluye

La primera versión de StockGo incluirá:

- Registro de usuarios.
- Inicio y cierre de sesión.
- Gestión de perfiles.

- Gestión de productos.
- Consulta de inventario.
- Registro de entradas.
- Registro de salidas.
- Historial de movimientos.
- Control de usuarios según su rol.
- Base de datos para almacenar la información.
- Comunicación con un servicio remoto.
- Integración de una API externa.
- Mecanismos de seguridad.
- Pruebas.
- Configuración de permisos.
- Generación de una versión release.

Estos elementos están alineados con los requerimientos mínimos establecidos para el proyecto.

No incluye inicialmente

Para evitar que el proyecto se vuelva demasiado grande, la primera versión **no incluirá**:

- Sistema de ventas completo.
- Facturación electrónica.
- Contabilidad.
- Gestión de proveedores avanzada.
- Sistema de nómina.
- Integración con sistemas externos de punto de venta.

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE

4. Requerimientos

4.1 Requerimientos funcionales

ID	Requerimiento
RF01	El sistema deberá permitir registrar usuarios.
RF02	El sistema deberá permitir iniciar sesión.
RF03	El sistema deberá permitir cerrar sesión.
RF04	El sistema deberá permitir consultar y modificar el perfil.
RF05	El administrador deberá poder registrar productos.
RF06	El administrador deberá poder modificar productos.
RF07	El administrador deberá poder eliminar productos.
RF08	Los usuarios autorizados deberán poder consultar el inventario.
RF09	Los usuarios autorizados deberán poder registrar entradas de

	productos.
RF10	Los usuarios autorizados deberán poder registrar salidas de productos.
RF11	El sistema deberá actualizar automáticamente las existencias después de cada movimiento.
RF12	El sistema deberá almacenar un historial de movimientos.
RF13	El sistema deberá identificar al usuario que realizó cada movimiento.
RF14	El Administrador podrá **administrar** los usuarios registrados (crear, consultar, modificar roles y desactivar cuentas).
RF15	El sistema deberá comunicarse con un servicio remoto.
RF16	El sistema deberá utilizar una API de terceros con una función relacionada con el inventario.
RF17	El sistema deberá asociar de forma obligatoria e inmutable cada registro de entrada o salida con la identidad del usuario que lo realizó.

4.2 Requerimientos no funcionales

ID	Requerimiento
RNF01	La aplicación deberá contar con autenticación de usuarios.
RNF02	La información deberá almacenarse de forma persistente.
RNF03	La comunicación con el servidor deberá realizarse de forma segura.
RNF04	La aplicación deberá controlar el acceso según el tipo de usuario.
RNF05	La interfaz deberá ser sencilla y fácil de utilizar.
RNF06	El sistema deberá responder correctamente ante errores de conexión.
RNF07	El proyecto deberá utilizar control de versiones mediante Git.
RNF08	La aplicación deberá contar con pruebas antes de generar la versión release.
RNF09	La aplicación deberá solicitar únicamente los permisos necesarios.
RNF10	El sistema deberá ser compilado, firmado y empaquetado en formato *release* optimizado listo para producción.

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE

5. Historias de usuario

Utilizaremos el formato:

> **Como [usuario], quiero [acción], para [beneficio].**

HU01 — Registro

Como usuario,
quiero registrarme en la aplicación,
para poder utilizar el sistema.

HU02 — Inicio de sesión

Como usuario registrado,
quiero iniciar sesión,
para acceder a las funciones correspondientes a mi cuenta.

HU03 — Gestión de productos

Como administrador,
quiero registrar, modificar y eliminar productos,
para mantener actualizado el inventario.

HU04 — Consulta de inventario

Como usuario,
quiero consultar los productos y sus existencias,
para conocer la cantidad disponible.

HU05 — Entrada de productos

Como empleado,
quiero registrar entradas de productos,
para actualizar las existencias cuando lleguen nuevos productos.

HU06 — Salida de productos

Como empleado,
quiero registrar salidas de productos,
para actualizar las existencias cuando se retiren productos.

HU07 — Historial

Como usuario,
quiero consultar el historial de movimientos,
para conocer las entradas y salidas realizadas.

HU08 — Administración de usuarios

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE

Como Administrador,
quiero gestionar las cuentas y roles de los usuarios,
para controlar los permisos y accesos al sistema.

HU09 — Perfil

Como usuario,
quiero consultar y modificar mi perfil,
para mantener actualizada mi información.

HU10 — API externa

Como usuario,
quiero utilizar una función basada en una API externa relacionada con los productos,
para obtener información que facilite la gestión del inventario.

6. Criterios de aceptación

ID	Historia	Criterios de aceptación
CA01	HU01	El usuario deberá proporcionar los datos requeridos y el sistema deberá crear su cuenta correctamente.
CA02	HU02	Si las credenciales son correctas, el usuario podrá acceder; si son incorrectas, se mostrará un mensaje de error.
CA03	HU03	El administrador podrá crear, modificar y eliminar productos.
CA04	HU04	El sistema deberá mostrar los productos registrados y su cantidad disponible.
CA05	HU05	Al registrar una entrada, la cantidad disponible deberá aumentar.
CA06	HU06	Al registrar una salida, la cantidad disponible deberá disminuir.
CA07	HU07	El sistema deberá mostrar los movimientos registrados junto con la información correspondiente.

CA08	HU08	Únicamente el administrador podrá administrar usuarios.
CA09	HU09	El usuario podrá consultar y actualizar sus datos permitidos.
CA10	HU10	La aplicación deberá obtener y mostrar correctamente la información proporcionada por la API externa.

7. Product Backlog

El documento solicita un **Product Backlog** en esta primera etapa.

	Historia	**Prioridad**	**Estimación**	**Sprint**
PB01	Registro de usuarios	Alta	5	Sprint 1
PB02	Inicio de sesión	Alta	5	Sprint 1
PB03	Cierre de sesión	Alta	2	Sprint 1
PB04	Gestión de perfil	Media	3	Sprint 2
PB05	Registrar productos	Alta	5	Sprint 2
PB06	Modificar productos	Alta	3	Sprint 2
PB07	Eliminar productos	Media	3	Sprint 2
PB08	Consultar inventario	Alta	5	Sprint 2
PB09	Registrar entradas	Alta	5	Sprint 3
PB10	Registrar salidas	Alta	5	Sprint 3
PB11	Actualizar existencias	Alta	5	Sprint 3
PB12	Historial de movimientos	Alta	5	Sprint 3
PB13	Administración de usuarios	Media	5	Sprint 4
PB14	Integración de API externa	Alta	8	Sprint 4
PB15	Servicio cloud	Alta	8	Sprint 4

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE

PB16	Seguridad y permisos	Alta	8	Sprint 5
PB17	Pruebas	Alta	8	Sprint 5
PB18	Preparación de release	Alta	5	Sprint 5

8. Metodología

Scrum
Se eligió el marco de trabajo Scrum debido a la necesidad de entregar incrementos de software funcionales de forma iterativa y adaptativa. Al tratarse de un desarrollo móvil integral con múltiples capas (Frontend móvil, Backend en Flask, Base de Datos MySQL y API de terceros), Scrum permite dividir el proyecto en 5 Sprints de 2 semanas cada uno. Esto facilita inspeccionar periódicamente la integración de cada componente, gestionar riesgos tempranamente (como fallos en endpoints o conectividad) y ajustar las funcionalidades clave según la prioridad del negocio.

Roles

Aunque el proyecto será realizado por una pareja, podemos organizar las responsabilidades de Scrum de la siguiente manera:

Rol	Responsabilidad
Product Owner	Definir prioridades y necesidades del producto.
Scrum Master	Organizar el trabajo, seguimiento y cumplimiento de los Sprints.
Equipo de desarrollo	Diseñar, programar, probar e integrar la aplicación.

Sprints propuestos

Sprint 1 — Base del sistema

- Registro.
- Login.
- Cierre de sesión.
- Estructura inicial.

Sprint 2 — Productos

- Perfil.
- Registrar productos.
- Modificar productos.
- Eliminar productos.
- Consulta de inventario.

Sprint 3 — Movimientos

- Entradas.
- Salidas.
- Actualización de existencias.
- Historial.

Sprint 4 — Conectividad

- Servicio cloud.
- API externa.
- Integración con backend.

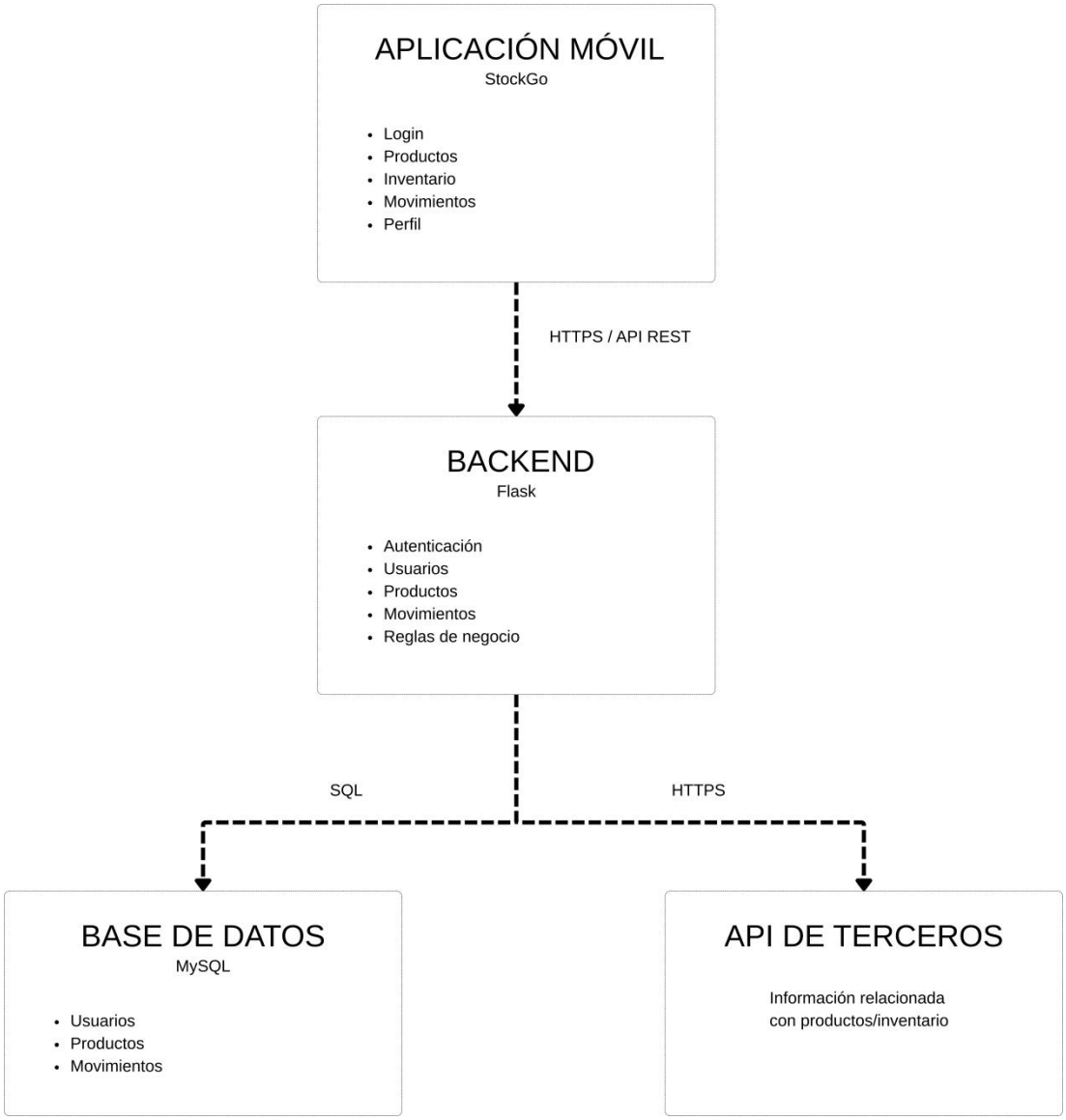
Sprint 5 — Seguridad y finalización

- Seguridad.
- Permisos.
- Pruebas.
- Correcciones.
- Release.

9. Arquitectura

Se seleccionó una arquitectura en 3 capas (Presentación, Lógica de Negocio y Datos) complementada con un patrón Cliente-Servidor de servicios RESTful. Esta elección garantiza la separación de responsabilidades: la aplicación móvil (Frontend) se enfoca en la experiencia de usuario y presentación; el servidor Backend (Flask) centraliza las reglas de negocio, la validación de tokens de seguridad y la lógica de inventario; mientras que la base de datos (MySQL) asegura la persistencia rigurosa de los datos. Esta desacoplada estructura permite escalar el sistema, mantener el código ordenado y evitar la exposición directa de la base de datos a los dispositivos móviles por motivos de seguridad..

Arquitectura general



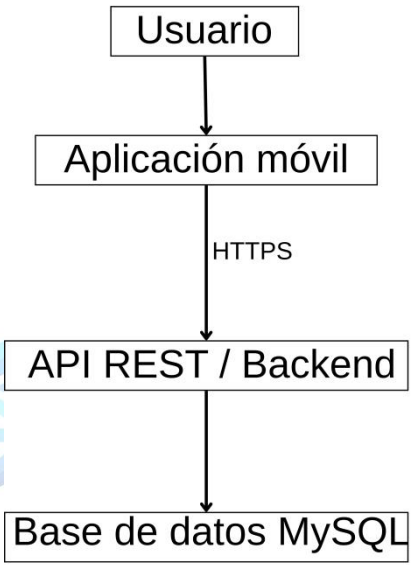
Componentes

Componente	Responsabilidad
Aplicación móvil	Interfaz mediante la cual los usuarios interactúan con StockGo.
Backend	Procesar las solicitudes, aplicar reglas de negocio y controlar la autenticación.
Base de datos	Almacenar usuarios, productos y movimientos.
API REST	Permitir la comunicación entre la aplicación móvil y el backend.

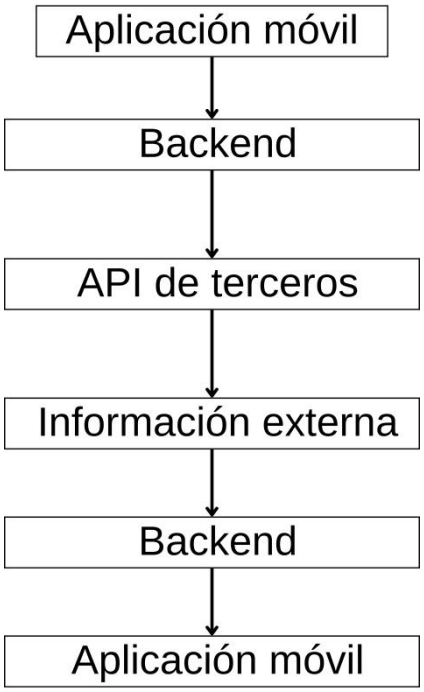
API de terceros	Proporcionar información externa utilizada por una función real de la aplicación.
Servicio cloud	Alojar los servicios necesarios para que la aplicación pueda comunicarse remotamente.

Comunicación

La comunicación principal será:



Cuando sea necesario utilizar la API externa:



El uso de un servicio remoto y una API de terceros es necesario porque el proyecto exige que la aplicación tenga comunicación con servicios externos.

10. Responsabilidades de cada capa

Capa de presentación

Se encargará de:

- Mostrar las pantallas.
- Recibir las acciones del usuario.
- Mostrar productos.
- Mostrar existencias.
- Mostrar movimientos.
- Mostrar mensajes de error o confirmación.

Capa de lógica de negocio

Se encargará de:

- Validar usuarios.
- Comprobar permisos.
- Procesar entradas y salidas.
- Calcular existencias.
- Validar productos.
- Gestionar las solicitudes de la aplicación.
- Comunicarse con APIs externas.

Capa de datos

Se encargará de:

- Guardar usuarios.
- Guardar productos.
- Guardar movimientos.
- Consultar información.
- Actualizar información.
- Eliminar información cuando corresponda.

ETAPA 2 — Patrones, framework y versionamiento

Para **StockGo**, la Etapa 2 debe demostrar que las decisiones técnicas tienen una razón real y no que se agregaron patrones solamente para cumplir con la actividad.

Tomando como base la arquitectura que ya definieron —aplicación móvil, backend Flask, MySQL y API REST—, yo utilizaría **Flutter + Dart** para la aplicación móvil y seleccionaría **Repository Pattern** y **Singleton** como patrones principales.

Patrones de diseño

Para StockGo se seleccionaron patrones que ayudan a organizar el código, separar responsabilidades y facilitar la comunicación entre la aplicación móvil y los servicios externos.

Patrón	Problema que resuelve	Lugar donde se utiliza	Justificación
Repository Pattern	Evita que las pantallas de la aplicación tengan que conocer directamente cómo se obtienen o almacenan los datos.	Aplicación móvil, entre la interfaz y los servicios/API.	Permite separar la lógica de presentación del acceso a datos. Si posteriormente cambia la forma de obtener información, las pantallas no necesitan modificarse completamente.
Singleton	Evita crear múltiples instancias innecesarias de un mismo servicio que debe ser compartido por la aplicación.	Servicios de comunicación HTTP y configuración de la aplicación móvil.	Permite centralizar elementos compartidos, como la configuración de las peticiones al backend y determinados servicios de comunicación.

Repository Pattern

El Repository Pattern será utilizado para centralizar el acceso a los datos que necesita la aplicación móvil.

Por ejemplo, cuando el usuario consulte el inventario, la pantalla no realizará directamente una petición HTTP. En su lugar, solicitará los datos al repositorio correspondiente.

El flujo será:

Pantalla → Repository → API REST → Backend → MySQL

Esto permitirá mantener separadas las responsabilidades y facilitar futuras modificaciones.

Singleton

El patrón Singleton será utilizado en servicios que deban contar con una única instancia compartida dentro de la aplicación, principalmente para la comunicación HTTP y determinadas configuraciones.

Por ejemplo, la aplicación podrá utilizar una instancia centralizada para realizar las peticiones al backend, evitando crear diferentes objetos de conexión para cada pantalla.

Framework de desarrollo móvil

Para desarrollar StockGo se seleccionará **Flutter** como framework de desarrollo móvil y **Dart** como lenguaje de programación.

Framework seleccionado

- Flutter

Lenguaje

- Dart

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE

Plataforma

Flutter permitirá desarrollar la aplicación móvil principalmente para **Android**, que será la plataforma objetivo de la primera versión de StockGo.

La elección de Android se relaciona con el objetivo del proyecto de generar posteriormente una versión **APK/AAB Release**, establecido desde la Etapa 1.

Lenguaje

Flutter utiliza **Dart**, un lenguaje desarrollado para crear aplicaciones con interfaces modernas y buen rendimiento.

Dart permitirá estructurar el proyecto de forma organizada mediante clases, servicios, modelos y repositorios, lo cual se adapta a la arquitectura propuesta para StockGo.

Capacidades

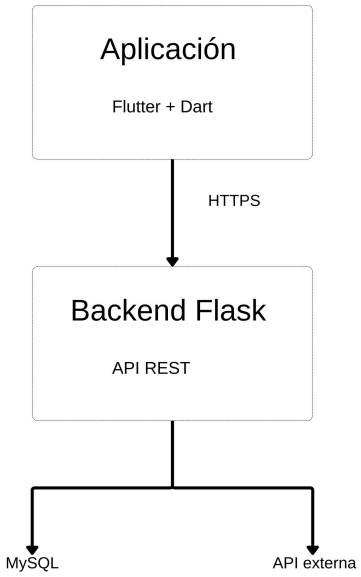
Flutter proporciona las herramientas necesarias para desarrollar las principales funciones de StockGo, entre ellas:

- Interfaces gráficas para dispositivos móviles.
- Formularios de registro e inicio de sesión.
- Navegación entre pantallas.
- Consumo de APIs REST.
- Manejo de información recibida desde el backend.
- Validación de datos.
- Manejo de sesiones.
- Integración con servicios externos.
- Manejo de permisos del dispositivo.
- Generación de versiones de producción.

Integración

Flutter se comunicará con el **Backend desarrollado con Flask** mediante solicitudes HTTP utilizando la API REST.

La comunicación propuesta será:



De esta manera, la aplicación móvil no tendrá acceso directo a la base de datos. Todas las operaciones pasarán por el backend, donde se aplicarán las reglas de negocio y los controles de seguridad.

Comunidad y documentación

Flutter cuenta con documentación oficial y una comunidad amplia de desarrolladores. Esto facilita encontrar información sobre componentes, paquetes, integración de servicios y solución de problemas durante el desarrollo.

Además, Dart y Flutter cuentan con herramientas específicas para pruebas, construcción de aplicaciones y generación de versiones para distribución.

Adecuación al proyecto

Flutter es adecuado para StockGo porque permite desarrollar una aplicación móvil completa utilizando una sola base tecnológica para la interfaz, mientras que el backend Flask puede encargarse de la lógica de negocio y la comunicación con MySQL y las APIs externas.

La combinación propuesta será:

Componente	**Tecnología**
Aplicación móvil	**Flutter**
Lenguaje móvil	**Dart**
Backend	**Python + Flask**
API	**REST**
Base de datos	**MySQL**
Control de versiones	**Git + GitHub**
Servicio remoto	**Cloud**
API externa	**API relacionada con productos/inventario**

Versionamiento

Para controlar los cambios realizados durante el desarrollo de StockGo se utilizará **Git** como sistema de control de versiones y **GitHub** como repositorio remoto.

El repositorio permitirá mantener un historial de los cambios, trabajar de manera organizada y recuperar versiones anteriores en caso de errores.

Repositorio

Se creará un repositorio denominado:

StockGo

Una estructura inicial propuesta será:

StockGo/

│

├── mobile/

│ └── Aplicación Flutter

│

├── backend/

```
│   └── API Flask
│
├── database/
│   └── Scripts MySQL
│
├── docs/
│   └── Documentación del proyecto
│
├── tests/
│   └── Pruebas
│
└── README.md
```

Esto permitirá separar los diferentes componentes del proyecto y mantener organizado el repositorio.

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE

Estrategia de ramas

Se utilizará una estrategia sencilla de ramas debido a que el proyecto será desarrollado por una pareja.

main

Contendrá las versiones estables del proyecto.

No se realizarán cambios experimentales directamente sobre esta rama.

develop

Contendrá la versión de desarrollo donde se integrarán las funcionalidades antes de considerarlas estables.

Ramas de funcionalidad

Para cada funcionalidad importante se podrá crear una rama:

feature/login

feature/inventario

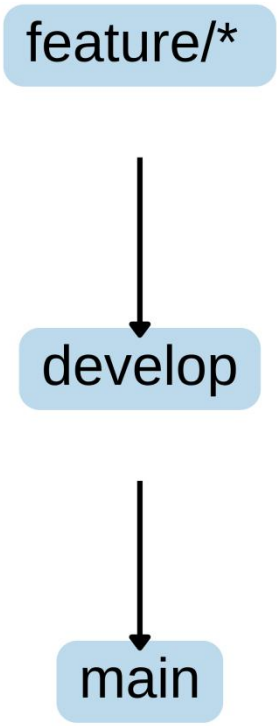
feature/productos

feature/movimientos

feature/api

feature/seguridad

El flujo será:



Cuando una funcionalidad sea terminada y probada, se integrará primero en develop. Después de comprobar que el sistema funciona correctamente, se podrá integrar en main.

Convención de commits

Para mantener un historial comprensible, los commits utilizarán una estructura sencilla:

tipo: descripción breve

Se utilizarán principalmente:

Tipo	Uso
feat	Nueva funcionalidad
fix	Corrección de un error
docs	Cambios en documentación
test	Creación o modificación de pruebas
refactor	Reorganización del código sin cambiar su funcionamiento
style	Cambios de formato o estilo

Ejemplos

feat: agregar inicio de sesión

feat: implementar registro de productos

fix: corregir actualización de existencias

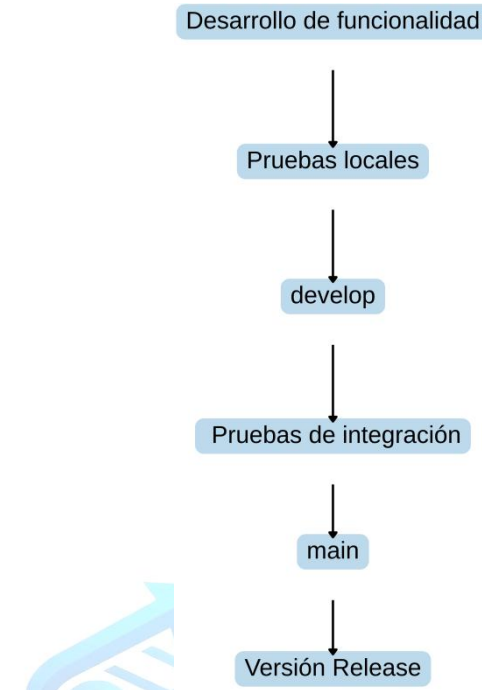
test: agregar pruebas para movimientos

docs: actualizar documentación de arquitectura

refactor: organizar servicio de inventario

Estrategia de integración

La integración del proyecto seguirá un flujo controlado para evitar que cambios incompletos lleguen a la versión estable.



Relación con Scrum

El versionamiento estará relacionado con los Sprints definidos en la Etapa 1.

Sprint	Trabajo	Rama
Sprint 1	Registro y login	feature/login
Sprint 2	Productos e inventario	feature/productos
Sprint 3	Entradas, salidas e historial	feature/movimientos
Sprint 4	Cloud y API externa	feature/api
Sprint 5	Seguridad, pruebas y release	feature/seguridad

De esta manera, cada Sprint podrá generar incrementos funcionales y los cambios realizados quedarán registrados mediante Git.

Resumen de decisiones técnicas

Elemento	Decisión
Framework móvil	**Flutter**
Lenguaje	**Dart**
Backend	**Flask / Python**
Base de datos	**MySQL**
Arquitectura	**3 capas + Cliente-Servidor REST**
Patrón 1	**Repository Pattern**
Patrón 2	**Singleton**
Control de versiones	**Git**
Repositorio remoto	**GitHub**
Rama estable	**main**
Rama de desarrollo	**develop**
Ramas de funcionalidades	**feature/***
Convención de commits	**feat, fix, docs, test, refactor, style**
Versión final	**APK/AAB Release**

ETAPA 3 — Seguridad, APIs y nube

En esta etapa se implementarán los mecanismos necesarios para conectar StockGo con servicios remotos, proteger la información y utilizar una API externa con una función relacionada directamente con la gestión del inventario.

La arquitectura continuará utilizando una aplicación móvil desarrollada con Flutter, un Backend desarrollado con Flask, una API REST y una base de datos MySQL alojada en un servicio cloud.

Seguridad

StockGo manejará información relacionada con usuarios, productos y movimientos de inventario. Por esta razón, se deberán implementar mecanismos de seguridad para evitar accesos no autorizados, modificación indebida de información y exposición de datos.

Información sensible

La información que deberá protegerse incluye:

- Credenciales de acceso de los usuarios.
- Datos personales de los usuarios.
- Roles y permisos.
- Información de productos.
- Cantidades disponibles en inventario.
- Historial de entradas y salidas.
- Identidad del usuario que realizó cada movimiento.

- Tokens de autenticación.
- Credenciales de conexión a la base de datos.
- Claves o configuraciones utilizadas para servicios externos.

Las contraseñas no deberán almacenarse directamente en la base de datos, sino mediante un mecanismo de **hash seguro**.

Amenazas identificadas

Entre las principales amenazas que podrían afectar a StockGo se encuentran:

1. Acceso de usuarios no autorizados.
2. Robo de credenciales.
3. Interceptación de información durante la comunicación.
4. Manipulación de solicitudes enviadas a la API.
5. Inyección SQL.
6. Acceso no autorizado a la base de datos.
7. Modificación de inventario por usuarios sin permisos.
8. Exposición de claves de servicios externos.
9. Robo o manipulación de tokens de sesión.
10. Pérdida de información por fallas del servicio.

Vulnerabilidades

Las principales vulnerabilidades que se deberán prevenir son:

- Contraseñas almacenadas de manera insegura.
- Comunicación sin cifrado.
- Falta de validación de datos recibidos.
- Falta de control de permisos.
- Consultas SQL construidas directamente con datos proporcionados por el usuario.
- Credenciales almacenadas directamente dentro del código.
- Endpoints accesibles sin autenticación.
- Falta de validación de tokens.
- Ausencia de respaldos de la información.

Matriz de seguridad

Elemento	Riesgo	Vulnerabilidad	Medida de protección
Login	Acceso de personas no autorizadas	Credenciales débiles o manejo inseguro de contraseñas	Autenticación, hash de contraseñas y tokens de sesión
Datos personales	Exposición o robo de información	Datos almacenados o transmitidos sin protección	HTTPS, control de acceso y almacenamiento seguro

API	Uso no autorizado o manipulación de solicitudes	Endpoints sin autenticación o validación	HTTPS, autenticación mediante tokens, autorización por roles y validación de datos
Base de datos	Robo o modificación de información	Acceso directo o consultas inseguras	Acceso únicamente mediante Backend, consultas parametrizadas, credenciales protegidas y respaldos
Inventario	Modificación no autorizada de existencias	Falta de control de permisos	Roles Administrador/Empleado y autorización en Backend
Movimientos	Alteración o eliminación indebida del historial	Falta de trazabilidad	Asociar cada movimiento al usuario responsable y controlar los permisos
API externa	Exposición de credenciales o uso indebido	Claves almacenadas directamente en el código	Variables de entorno y validación de solicitudes
Servidor cloud	Pérdida o acceso no autorizado a información	Configuración incorrecta del servicio	Credenciales protegidas, acceso restringido y configuración segura

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE

Mecanismos de protección

Para reducir los riesgos identificados se implementarán los siguientes mecanismos:

Autenticación

Los usuarios deberán iniciar sesión mediante sus credenciales antes de acceder a las funciones protegidas de StockGo.

Autorización por roles

El Backend verificará el rol del usuario antes de permitir determinadas operaciones.

Por ejemplo:

Administrador

- Crear productos.
- Modificar productos.
- Eliminar productos.

- Administrar usuarios.
- Consultar movimientos.

Empleado

- Consultar inventario.
- Registrar entradas.
- Registrar salidas.
- Consultar movimientos.
- Modificar su perfil.

HTTPS

La comunicación entre la aplicación móvil y el Backend deberá realizarse mediante **HTTPS**, evitando que las credenciales y demás información viajen sin cifrado.

Protección de contraseñas

Las contraseñas no serán almacenadas directamente. Se utilizará un algoritmo de hash seguro para almacenarlas.

Validación de datos

El Backend validará la información recibida antes de procesarla.

Protección contra SQL Injection

Las consultas a MySQL utilizarán consultas parametrizadas para evitar que datos introducidos por el usuario puedan modificar las consultas SQL.

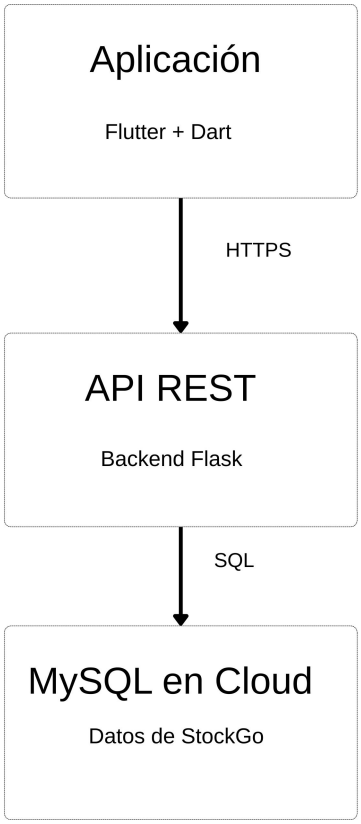
Variables de entorno

Las credenciales de la base de datos, claves secretas y configuraciones sensibles no se almacenarán directamente dentro del código fuente.

Servicios Cloud

StockGo deberá utilizar un servicio remoto para permitir que la aplicación móvil se comunique con el Backend y la base de datos sin depender únicamente del equipo local de desarrollo.

La arquitectura propuesta será:



El servicio cloud permitirá que el Backend y la base de datos puedan estar disponibles remotamente para la aplicación móvil.

Datos almacenados en la nube

La base de datos almacenará principalmente:

- Usuarios.
- Roles.
- Productos.
- Existencias.
- Entradas.
- Salidas.
- Historial de movimientos.

El acceso a la base de datos se realizará a través del Backend y no directamente desde la aplicación móvil.

API de terceros

Para cumplir con el requisito de integración de una API externa, StockGo utilizará **Open Food Facts**.

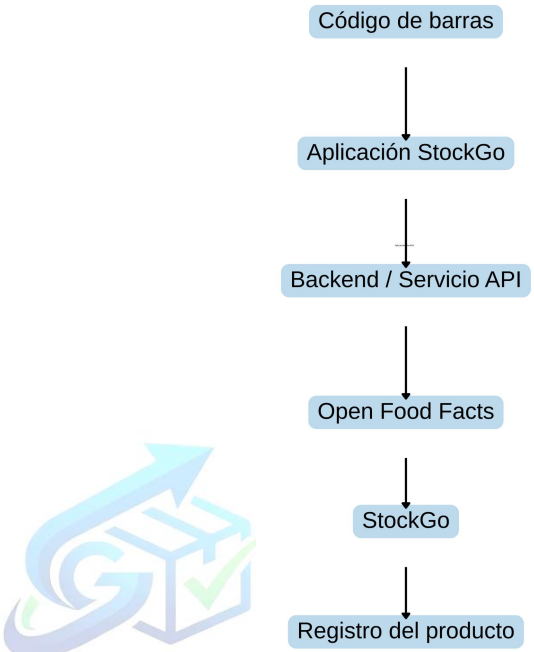
La API permitirá consultar información de productos alimenticios utilizando su **código de barras**.

Esto tiene una función real dentro de StockGo porque permitirá agilizar el registro de productos.

Función dentro de StockGo

Cuando el administrador registre un nuevo producto, podrá introducir o escanear su código de barras.

StockGo enviará el código a la API externa:



La información obtenida podrá utilizarse para facilitar el llenado de datos como:

- Nombre del producto.
- Marca.
- Categoría.
- Imagen del producto, cuando esté disponible.
- Información adicional proporcionada por la API.

El usuario podrá revisar la información antes de guardar el producto en el inventario.

Justificación de la API externa

La integración de Open Food Facts no se realizará únicamente para demostrar que se puede consumir una API.

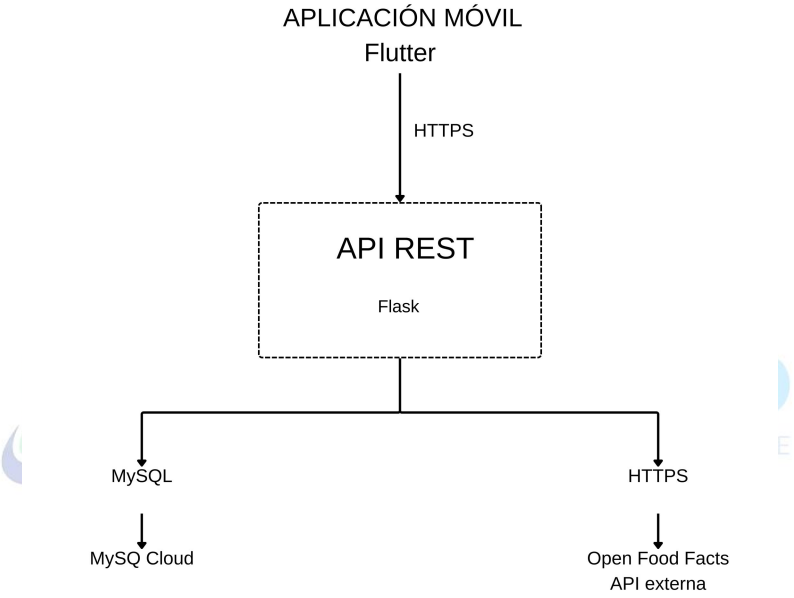
Su utilización tendrá una función específica dentro del sistema: reducir el tiempo necesario para registrar productos en el inventario.

Por ejemplo, en lugar de que un empleado tenga que escribir manualmente el nombre y otros datos de un producto, el sistema podrá consultar la información disponible utilizando su código de barras.

Esto hace que la API externa esté relacionada directamente con una operación real de StockGo.

Comunicación entre los componentes

La comunicación general de StockGo será:

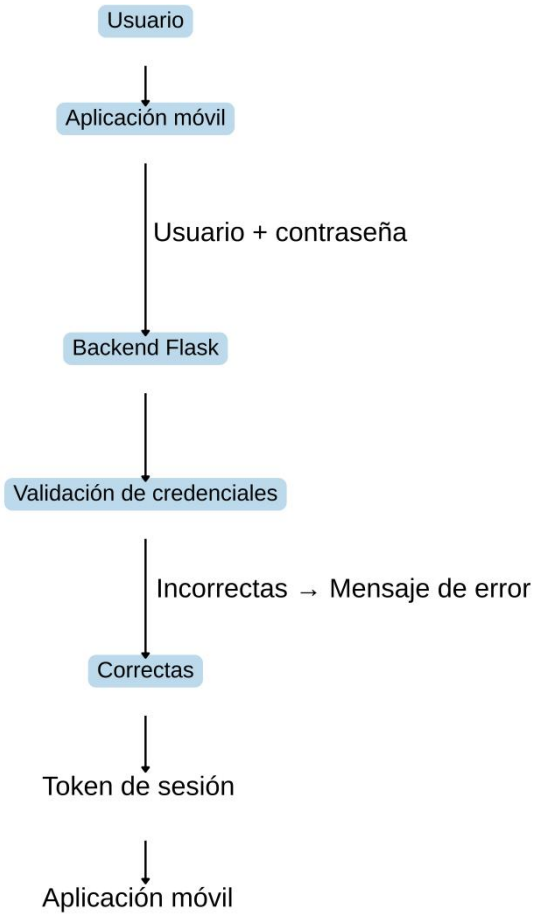


El Backend funcionará como intermediario entre la aplicación móvil, la base de datos y la API externa.

Esto permite evitar que la aplicación móvil tenga acceso directo a la base de datos y permite centralizar las reglas de seguridad y negocio.

Flujo de autenticación

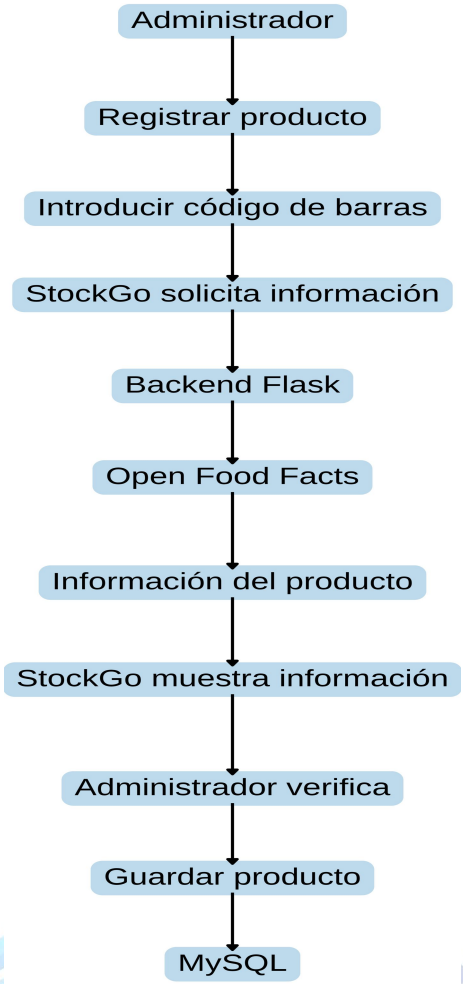
El inicio de sesión seguirá un flujo similar al siguiente:



Posteriormente, las solicitudes que requieran autenticación deberán incluir el mecanismo correspondiente para que el Backend pueda identificar al usuario.

Flujo de registro de producto mediante API

Un ejemplo del funcionamiento de la API externa será:



De esta manera, la API externa se integra directamente con una funcionalidad del sistema y aporta un beneficio al usuario.

Relación con la arquitectura del proyecto

La Etapa 3 mantiene la arquitectura definida anteriormente:

Componente	Función
Flutter	Interfaz móvil y comunicación con el Backend
API REST	Punto de comunicación entre aplicación y servidor
Flask	Lógica de negocio, autenticación y autorización
MySQL Cloud	Persistencia de usuarios, productos y movimientos
Open Food Facts	Consulta externa de información de productos
HTTPS	Protección de la comunicación
Git/GitHub	Control de versiones

StockGo
GESTIÓN DE INVENTARIO INTELIGENTE